

Research & Development Report

AI Healthcare Operations & Administration Agent

Prepared by: Alam Wazir
Submitted to: Project Lead
Date: September 2026

This report documents my own research and development process on the assigned project brief. It covers how I interpreted the requirement, the alternatives I compared for each major technology decision and why I settled on a specific choice, the risks I identified, the assumptions I am making, and the implementation plan I intend to follow. The goal of this document is to show that I understand the system deeply enough that implementation becomes execution, not further design.

1. Research Objective & Method

The brief I was given describes an AI-powered healthcare operations and administration agent - an assistant that automates administrative work (appointments, billing, insurance, reports, policy lookup) through natural-language conversation, without touching diagnosis or treatment. My task was to independently research the right way to build this, not just accept the brief's tool list at face value.

My research process had four parts:

- **Requirement breakdown** - separating what the user-facing agent must do from what the backend must guarantee (security, correctness).
- **Technology comparison** - for every major component (orchestration, LLM hosting, API layer, database, vector store, auth), I compared at least one realistic alternative before accepting the brief's suggested choice.
- **Risk identification** - where an LLM-driven system like this is most likely to fail or be exploited, and how the design should defend against it.
- **Implementation planning** - translating the architecture into an actual phase-by-phase, testable build order with rough effort estimates

Where I fully agree with the brief's technology choice, I explain *why* it's the right call rather than repeating it. Where I see a genuine trade-off, I say so and recommend a direction.

2. My Understanding of the System

In my own words: this is not a chatbot that answers questions from what it "knows." It is a **controlled decision-and-execution system** where a language model is only allowed to do one thing - decide which pre-approved operation to run next and with what arguments. Everything that actually touches data (booking a slot, reading a balance, sending a message) happens in plain, testable application code that the LLM cannot bypass or talk its way around.

The reason this distinction matters, in my understanding: an LLM is a reasoning engine, not a transaction engine. It has no reliable concept of "this operation has actually committed" or "this user is genuinely who they claim to be." So the architecture has to put those guarantees outside the model entirely - in schema validation, in a permission engine, and in a database transaction. This is the single idea the rest of my technology choices below are built around.

Three things convinced me this has to be an *agent* and not a simple pipeline:

- The number of tools is too large (19+) for a single prompt to reliably choose from without a structured selection/validation loop.
- Real workflows are multi-turn - a booking spans several messages, so the system needs a state object that survives across turns, not just a stateless request/response.
- Some outcomes are conditional on runtime data (is the slot still free right now?), which means the plan has to be able to branch and retry, not just execute a fixed script.

3. Technology Research & Comparisons

For each core component I looked at the option the brief proposes against at least one realistic alternative, and recorded why I would proceed with a specific choice.

3.1 Agent Orchestration: LangGraph vs. Plain Function-Calling vs. CrewAI/AutoGen

Option	Strength	Weakness
Plain function-calling loop (manual while-loop around an LLM API)	Simple to start	No built-in state persistence, no branching/approval primitives - I'd end up rebuilding LangGraph's core features by hand
CrewAI / AutoGen (multi-agent frameworks)	Good for role-playing multiple cooperating agents	Designed around agent-to-agent conversation, not a single deterministic tool-execution graph with approval gates - heavier than this use case needs
LangGraph (chosen)	Native state graph, conditional edges, explicit interrupt/approval points, checkpointing for multi-turn workflows	Steeper learning curve than a plain loop

My conclusion: LangGraph is the right choice specifically because appointment booking needs a pause-and-resume point (confirmation) and a hard branch (slot available vs. conflict) as first-class graph features - not something I want to hand-roll with flags and if-statements that get harder to test as workflows grow.

3.2 LLM Hosting: Groq (Llama/Qwen) vs. OpenAI/Claude API vs. Self-Hosted

Option	Strength	Weakness
OpenAI / Anthropic hosted API	Strong reasoning quality, minimal ops	Per-token cost at scale; sending patient-adjacent admin data to a third-party API needs contractual review
Self-hosted open model (own GPU)	Full data control	Infra cost and MLOps burden not justified for a portfolio-stage build
Groq + Llama/Qwen (chosen)	Very low inference latency, open-weight models, good enough reasoning for structured tool-selection (not creative writing)	Slightly less nuanced reasoning than top closed models for ambiguous edge cases

My conclusion: tool-selection and slot-filling is a narrower task than open-ended chat - it doesn't need the most powerful model, it needs a fast, consistent one. Groq's inference speed matters more here than raw reasoning ceiling, especially across multi-tool workflows where several calls happen per user turn.

3.3 API Framework: FastAPI vs. Flask/Django

Option	Strength	Weakness
Flask	Minimal, familiar	No native async, no built-in request/response validation - I'd add Pydantic and async support manually
Django	Batteries-included (admin, ORM)	Heavier than needed; this project's persistence is a handful of tables, not a full CMS-style app
FastAPI (chosen)	Native async (matters for calling an LLM + multiple tools per request), Pydantic validation built in, auto-generated OpenAPI docs for the boss/team to review endpoints	Smaller ecosystem than Django for admin-style tooling

My conclusion: async support isn't a nice-to-have here - a single chat request can trigger several sequential/parallel tool + I/O calls, and blocking synchronous handlers would hurt latency under any real load.

3.4 Database: PostgreSQL vs. MongoDB / NoSQL

Option	Strength	Weakness
MongoDB (document store)	Flexible schema, fast to prototype	Appointments, billing and audit logs are inherently relational (foreign keys, joins, transactional consistency) - enforcing that in a document store means reimplementing relational guarantees in application code
PostgreSQL (chosen)	ACID transactions (critical for the appointment-conflict re-check), mature RBAC-friendly row-level patterns, pgvector extension covers RAG in the same engine	Slightly more upfront schema design than a document store

My conclusion: the single most safety-critical operation in this whole system is "re-check availability, then write, atomically." That's a textbook transactional requirement - PostgreSQL is the right tool, not a preference.

3.5 Vector Store: pgvector vs. Qdrant vs. Pinecone

- **Pinecone** - managed, easy, but an external paid dependency for what is initially a small internal policy corpus.
- **Qdrant** - strong standalone vector DB if the policy corpus grows large or needs advanced filtering.
- **pgvector (my starting choice)** - keeps policy embeddings in the same PostgreSQL instance as everything else, so there's one database to operate, back up and secure during Stage A/B. I'd only move to Qdrant if the policy document volume or query load actually outgrows it - no need to add a second data store on day one.

3.6 Authentication: JWT vs. Server-Side Sessions

- **Server-side sessions** - simpler to revoke instantly, but needs sticky state on the API server, which fights against FastAPI's stateless/async model and complicates horizontal scaling.

- **JWT (chosen)** - stateless, carries role/scope claims the permission engine can read directly without a DB round-trip on every request. I'll pair it with short expiry + refresh tokens so the usual JWT revocation weakness is mitigated.

4. Architecture Reasoning - Why the Design Is Shaped This Way

4.1 Why build the agent before the backend (Stage A -> Stage B)

My reasoning: authentication and RBAC are a fixed, well-understood engineering problem - they don't need iteration once designed correctly. Agent reasoning quality (does it pick the right tool, does it ask for the right missing field, does it handle a conflict gracefully) is the actually uncertain part of this project. Building security around an agent whose reasoning isn't proven yet means re-testing the security layer every time the agent logic changes. Proving the agent first, then wrapping a fixed security boundary around a frozen version, means each layer is tested exactly once and stays tested.

4.2 Why authorization must sit outside the LLM

This is the part of the brief I most strongly agree with, and my own reasoning independently arrives at the same place: an LLM's "decision" is a probability distribution over tokens, not a security guarantee. If permission logic were expressed only as a system-prompt instruction ("don't let patients see other patients' data"), a sufficiently adversarial or just confusing conversation could get the model to violate it. The fix isn't a better prompt - it's a plain deterministic function (authorize(user, tool, scope, args) -> ALLOW/DENY) that runs after the model proposes a tool call and before any tool executes. The LLM can request anything; only the backend can grant it.

4.3 Why the final-availability re-check matters

Between the moment the agent shows a user available slots and the moment they confirm, real time passes and another user could book the same slot. Any booking system that doesn't re-check immediately before the write is vulnerable to a race condition that silently double-books a doctor. This is a correctness issue, not an AI issue - it would matter even with a fully deterministic, non-AI booking form.

5. Core Flow (My Working Model)

This is how I now mentally trace every request through the system before writing any code:

User message

- > LangGraph loads/creates conversation state
- > LLM classifies intent + extracts known fields
- > LLM proposes ONE tool call (never raw SQL/code)
- > schema validates arguments
- > permission engine: ALLOW / DENY (deterministic, outside the LLM)
- > DENY -> safe refusal, no data leak, audit event
- > ALLOW -> service executes -> normalized result -> graph state
- > agent decides: more tools needed? missing info? confirmation/approval needed? done?
- > loop until done
- > final answer built ONLY from verified tool results

For write operations (create/cancel/reschedule) the loop additionally requires: explicit user confirmation -> re-check the resource immediately before writing -> write -> audit_log -> communication. I treat this confirmation-plus-recheck pattern as non-optional for every consequential action, not just booking.

6. Risks I Identified

Risk	Mitigation I'd Build In
Prompt injection via patient-provided free text (e.g. inside a message the agent later summarizes)	Never let tool arguments be built by directly interpolating raw user text into privileged fields (e.g. patient_id) - always resolve identity from the authenticated session, not from what the user types.
LLM hallucinating a successful booking when a tool call actually failed or timed out	Final answer generation is only allowed to reference the structured tool_results list in state - never allowed to narrate an action it didn't see a successful result for.
Race condition double-booking a slot	Mandatory re-check-then-write inside a single DB transaction immediately before appointment_create.
Over-broad tool access letting a lower-privileged role reach a higher-privileged tool indirectly (e.g. through a multi-tool workflow)	Every individual tool call is authorized independently - a workflow can't "borrow" permission from an earlier allowed step.
Vector store returning stale or wrong policy text	Version policy documents; policy_rag responses cite the source/version so incorrect retrieval is traceable and fixable.
Clinical questions slipping through as if they were administrative	Explicit escalation intent classification before any other tool routing, with a hard rule: no clinical answer is ever generated, only escalated.

7. Assumptions & Open Questions for Review

Assumptions I'm proceeding on unless corrected:

- Stage A uses fully synthetic data - no real patient information touches the system before Stage B security is in place.
- Calendar, insurance and communication providers start as mock services with clean interfaces, to be swapped for real integrations later without touching agent/tool logic.
- A single LLM (via Groq) handles both intent classification and tool selection - I'm not planning a separate classifier model unless evaluation shows the single-model approach misclassifies too often.
- Bulk communication and sensitive report distribution require human approval by default; I'll treat the exact approval threshold (e.g. "bulk" = more than N recipients) as configurable, not hardcoded.

Questions I'd like confirmed before Phase 1 starts:

- Is there a real target deployment environment already decided (cloud provider, on-prem), or is Docker Compose on a single host acceptable for the portfolio/demo stage?
- Should audit logs be append-only/immutable (e.g. write-once table or external log store) for compliance, or is a normal PostgreSQL table sufficient for this stage?
- Is there an existing house style/policy document format I should model the policy_rag corpus on, or do I create the synthetic policy documents from scratch?

8. My Proposed Implementation Plan

Phase	Deliverables	Notes
Phase 1 - Foundation	Tool contracts/schemas, synthetic data set, every tool implemented + unit-tested	Highest priority - nothing else can be validated without this
Phase 2 - Agent Core	LangChain + Groq wiring, LangGraph state/planner, tool-selection loop	Depends on Phase 1 tools existing and passing unit tests
Phase 3 - Core Workflows	Stateful booking, reschedule, cancel, billing/insurance/analytics, policy RAG, web search, communication/reports/escalation	Built and tested workflow-by-workflow, booking first since it's the most complex state machine
Phase 4 - Hardening & Freeze	Retries, validation, safe-failure handling, full agent evaluation, freeze stable version	Gate before backend work starts - I will not begin Phase 5 until Phase 4 tests pass
Phase 5 - Secure Backend	FastAPI, PostgreSQL persistence + migrations, JWT auth, RBAC + data scopes, approvals, audit logging, Redis	Wraps the frozen agent - internal agent logic does not change here
Phase 6 - Deployment	Dockerize, integration/security tests, deploy, end-to-end verification	Final gate before calling the build production-ready

I'm sequencing it this way deliberately: each phase produces something independently testable before the next phase depends on it, so a mistake in an early phase is caught before it's built on top of.

9. Conclusion

Through this research I've confirmed that the brief's architecture - LangGraph + LangChain agent, FastAPI/JWT/RBAC backend, PostgreSQL + pgvector, deterministic permission engine outside the LLM - is the correct shape for this problem, and in each case I can independently justify why, not just repeat it. The one idea everything else follows from is: **the model proposes, the application disposes**. Every design decision above - from choosing LangGraph's interrupt states, to the final-availability re-check, to keeping authorization deterministic and outside the model - exists to enforce that one sentence.

With this understanding in place, I'm ready to begin Phase 1 - finalizing tool contracts and building the synthetic dataset - pending your confirmation on the open questions in Section 7.